Are LLM Written Unit Tests Flaky?

Project Proposal, ECE 382V Software Testing in the Era of Nondeterminism, Fall 2026

Shaunak Kapur (sk62288)
The University of Texas at Austin
Austin, Texas, USA

Manas Pathak (map7848)
The University of Texas at Austin
Austin, Texas, USA

1 PROBLEM

A lot of developers now let large language models write their unit tests, through tools like GitHub Copilot and ChatGPT and research tools like TestPilot [6]. These pipelines usually generate a test, run it once, and keep it if it passes. A test that passes once is not always a test that passes every time. It can depend on the order the tests run in, on the iteration order of a set or dict, on the wall clock, on random numbers, or on how busy the machine is. Tests like this are flaky, and they waste developer time and make people stop trusting their test results [4]. Almost every large flaky test study looks at tests written by people [1, 3], so we do not know whether LLM written tests are more or less flaky, what makes them flaky when they are, or whether the run once filter that most pipelines use is enough to keep flaky tests out.

We plan to answer four questions. **RQ1.** How often are LLM written tests flaky compared with the human written tests of the same library? **RQ2.** What are the root causes when they are? **RQ3.** Can simple risky patterns in test code, like real sleeps or reading the clock without freezing it, predict which tests flip? **RQ4.** Does the way the tests are generated matter, and can a cheap fix in the pipeline remove the flaky ones?

2 PROPOSED SOLUTION

We will run an empirical study and then try a mitigation. We picked 12 small pure Python libraries that have their own pytest suites, need no network, and run in under a minute. Half are plain utility libraries (toolz, tabulate, tinydb, humanize, cachetools, schedule) and half were chosen because they use time, randomness, threads, or files (tenacity, filelock, boltons, more-itertools, arrow, pybreaker).

For each library we generate test suites with the same prompt under several setups. One is an agent that runs its tests and fixes the failing ones, which is close to how TestPilot works. Two are one shot setups that write the tests without ever running them, one with a larger model (Claude Sonnet) and one with a smaller model (Claude Haiku). Later we will add a non Claude model and a plain prompt that does not point at the time or randomness features of the library. For the one shot suites we apply the same single run filter a real pipeline uses, but we keep the rejected tests so we can see what the filter throws away.

We then rerun every generated suite and the human written suite of each library many times while changing the test order and hash seed (pytest-randomly and `PYTHONHASHSEED`), the time zone (10 zones from UTC−11 to UTC+14), and the CPU load (one busy process per core). A test is flaky if it both passes and fails. For every flaky test we do targeted reruns, alone and inside its suite under each change separately, to find what triggers it, and we label the root cause with the categories from Luo et al. [3]. We also wrote a static scanner that flags risky patterns in each test (real sleeps, unfrozen clock reads, assertions on elapsed time, unseeded randomness, threads, subprocesses, and asserting the order of a set). For the mitigation we will compare a prompt that warns the model about these patterns and a filter that reruns each new test a few times under different conditions before keeping it.

We already wrote the rerun scripts and ran a small pilot on a couple of libraries. It found an LLM written test that schedules a job until two hours from now written as a clock time, which fails only after 10 PM, and a test that asserts two random retry delays add up to at least 10 ms. Both passed a single run, so a real pipeline would have kept them. The pilot also found human tests that only fail in certain orders because an earlier test leaves something in a global registry, which is the pattern iDFlakies [2] looks for.

3 EVALUATION

For RQ1 we will report the flaky rate of each suite and each generation setup with 95% confidence intervals, using the human suites of the same libraries as the baseline. For RQ2 we will report the root cause of every flaky test and which change exposed it. For RQ3 we will measure the precision and recall of the static scanner against the tests that really flipped. For RQ4 we will compare the setups, and for each mitigation we will count the flaky tests it keeps, the good tests it throws away, and the extra run time, and check line coverage with pytest-cov so a fix cannot win just by deleting tests.

Because flaky tests are rare, we expect to need a lot of reruns. Our main resources are our laptops, GitHub Actions on a private repository (2,000 free minutes per month on 2 vCPU Linux runners, which also gives us a slower, CI like machine and lets long runs finish without a laptop staying on), access to Claude through Claude Code, and a small amount of API credit for a second model family, which we expect to be under $20. The tooling is Python 3.12 with pytest, pytest-randomly, pytest-timeout, pytest-repeat, freezegun, and pytest-cov.

4 INTERMEDIATE STEPS 1 (BY OCTOBER 23)

Finish the subject list and generate every suite under the agentic and both one shot setups. Run the full rerun experiment on all suites with all three kinds of changes, report flaky rates for RQ1, and label the root cause of every flaky test for RQ2. Finish the first version of the static scanner.

5 INTERMEDIATE STEPS 2 (BY NOVEMBER 6)

Rerun every test the scanner flags at least 1,000 times on the GitHub Actions Linux runners, with and without CPU stress, to see whether the risky tests flip on a slower machine. Add the non Claude model and the plain prompt. Measure scanner precision and recall and the

coverage of every suite, and search the generated suites for order dependent tests in the style of iDFlakies [2].

6 INTERMEDIATE STEPS 3 (BY NOVEMBER 20)

Run the mitigation experiments, the warning prompt and the multi run filter, and compare them with the single run filter on flaky tests kept, good tests lost, coverage, and cost. For the timing tests that rarely fail we will also try delay injection in the style of FlakeRake [5]. Start the final report in the ACM format.

7 RELATED PAPERS

Luo et al. [3] give the root cause categories we use, and Gruber et al. [1] show what flakiness looks like in Python projects. TestPilot [6] is the generate and filter pipeline we imitate, and it keeps tests after one passing run. iDFlakies [2] detects order dependent tests by rerunning in random orders, and FlakeRake [5] reproduces rare timing failures by inserting delays. Parry et al. [4] survey the field.

8 NOTES

The progress report dates above follow the Canvas due dates. All code, prompts, generated tests, and raw results are kept in a private GitHub repository that we can share with the instructor.

REFERENCES

[1] Martin Gruber, Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2021. An Empirical Study of Flaky Tests in Python. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 148–158.

[2] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 312–322.

[3] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 643–653.

[4] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A Survey of Flaky Tests. *ACM Transactions on Software Engineering and Methodology* 31, 1 (2021).

[5] Shanto Rahman, Aaron Massey, Wing Lam, August Shi, and Jonathan Bell. 2024. Automatically Reproducing Timing-Dependent Flaky-Test Failures. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*.

[6] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105.